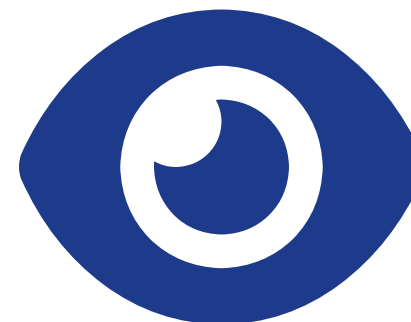


Aula 07: Views como Ferramenta de Segurança e Simplificação (DCL)

Controle de Acesso Baseado
em Papéis (RBAC) na Prática



Abertura: O Conceito de View (Tabela Virtual)

🎯 Definição Formal

Uma **View** é uma **tabela virtual** baseada no resultado de uma consulta ``SELECT``. Ela não armazena dados próprios; sempre consulta os dados da(s) tabela(s) base subjacente(s) no momento em que é acessada.

🗄️ A Relação com a Tabela Base

A View é dinâmica. Se os dados na tabela base mudam, os dados na View mudam automaticamente. Ela é apenas uma **janela** para os dados.

🎯 Propósito Principal (Revisão)

🔗 Simplificação de Consultas

Esconder a complexidade de ``JOINS`` múltiplos, filtros longos (``WHERE``) ou cálculos complexos. O usuário final consulta a View como se fosse uma tabela simples.

🔄 Consistência de Dados

Garantir que todos os usuários, analistas e aplicações vejam o **mesmo conjunto de dados** e a mesma lógica de negócio, evitando erros de consulta.

Views como Ferramenta de Segurança (O Conceito Chave)

A View é a principal maneira de aplicar o **Princípio do Mínimo Privilégio (PoLP)** na leitura de dados, atuando como um **Filtro de Segurança**.

Filtro Vertical (Restrição de Colunas)

Você pode criar uma View que seleciona apenas as **colunas não sensíveis** (Nome, ID) da tabela base, omitindo colunas sensíveis (CPF, Renda, Salário).

Como Funciona: Você concede permissão `SELECT` apenas na View, e não na tabela base. O usuário só consegue acessar as informações expostas pela View.

Vantagem: Mesmo que o usuário tente acessar a tabela base, ele não terá permissão para ver as colunas sensíveis, pois a permissão foi dada apenas na View filtrada.

Filtro Horizontal (Restrição de Linhas)

Você pode incluir uma cláusula `WHERE` na definição da View para restringir o acesso a **linhas específicas** (registros).

Exemplo: Uma View que mostra apenas clientes com `status = 'ativo'`. O usuário que acessa essa View não tem como ver os clientes `inativos` ou `pendentes`.

Vantagem: O usuário só vê o subconjunto de dados que é estritamente necessário para sua função, reforçando o PoLP em nível de registro.

Cenário Prático: Tabela Clientes e Dados Sensíveis

Tabela Base: Clientes

Estrutura da Tabela Clientes			
id_cliente	nome	cpf	status
1	João Silva	123.456.789-00	ativo
2	Maria Santos	987.654.321-11	ativo
3	Pedro Costa	555.666.777-88	inativo

CPF é um dado sensível (PII - Personally Identifiable Information)

Problema: Acesso Irrestrito

Um usuário de baixo privilégio (ex: relator_visitas) precisa ver a lista de clientes **ativos** para gerar um relatório, mas **não** deve ter acesso ao CPF (dado sensível).

Se você conceder `SELECT` na tabela base Clientes, o usuário terá acesso a **todas as colunas**, incluindo o CPF. Isso viola o **Princípio do Mínimo Privilégio (PoLP)**.

Solução: Criar uma View

Criar uma **View** que:

- Seleciona apenas as colunas **não sensíveis** (id_cliente, nome, status)
- Filtra apenas clientes com `status = 'ativo'`
- Concede `SELECT` **apenas na View**, não na tabela base

Resultado: O usuário vê apenas o que precisa, sem acesso a dados sensíveis. **PoLP aplicado com sucesso!**

Etapa 1: Criação da View (CREATE VIEW)



****Objetivo**:** Criar uma View que simplifique a consulta, mostrando apenas clientes com `status = 'ativo'`, e omitindo o campo sensível `cpf`.

</> Comando DDL (CREATE VIEW)

```
CREATE VIEW Clientes_Ativos_Simples AS
SELECT id_cliente, nome, status
FROM Clientes
WHERE status = 'ativo';
```

****O que este comando faz**:**

- ✓ Cria uma View chamada `Clientes\_Ativos\_Simples`
- ✓ Seleciona apenas 3 colunas: `id\_cliente`, `nome`, `status` (omitindo o CPF sensível)
- ✓ Filtra apenas os clientes com `status = 'ativo'` (filtro horizontal)
- ✓ A View é dinâmica: se os dados mudam na tabela base, a View reflete as mudanças automaticamente

Etapa 2: Validação da View e Criação do Usuário de Teste

✓ Validação da View (SELECT)

Execute um `SELECT` na View recém-criada para verificar se apenas os clientes ativos e os campos não sensíveis são retornados.

```
SELECT * FROM Clientes_Ativos_Simples;
```

Resultado Esperado: Apenas clientes com `status='ativo'` e sem a coluna `cpf` (dados sensíveis omitidos).

Validação: Se a View retorna os dados corretos, você pode prosseguir para a Etapa 3 (GRANT).

👤+ Criação do Usuário de Teste (CREATE USER)

Crie um novo usuário de ****baixo privilégio**** (`relator\_visitas`) que só precisa ver a lista de clientes ativos. Este usuário será usado para testar a segurança da View.

```
CREATE USER 'relator_visitas'@'localhost' IDENTIFIED BY 'senha_teste';
```

Resultado Esperado: Usuário `relator\_visitas` criado com sucesso. Ele ainda não tem nenhuma permissão.

Próximo Passo: Na Etapa 3, você concederá permissão `SELECT` ****apenas na View****, não na tabela base.

Etapa 3: Concessão de Privilégio APENAS na View (GRANT)

🔑 Comando DCL (GRANT)

Objetivo: Conceder acesso `SELECT` **apenas** à View recém-criada, e negar qualquer acesso à tabela base `Clientes`.

```
GRANT SELECT ON nome_do_banco.Clientes_Ativos_Simples
TO 'relator_visitas'@'localhost';
```

✓ **Resultado Esperado:** O usuário `relator\_visitas` agora pode executar `SELECT` **apenas** na View `Clientes\_Ativos\_Simples`.

🚫 Importância: Negar Acesso à Tabela Base

⚠️ Por que Não Conceder GRANT na Tabela Base?

Se você concedesse `SELECT` diretamente na tabela `Clientes`, o usuário poderia ver **todas as colunas**, incluindo o CPF (sensível), e **todas as linhas**, incluindo os clientes inativos. Isso viola o PoLP.

🛡️ A Segurança da View

A View atua como um **intermediário seguro**. O usuário acessa a View, não a tabela base. A View filtra as colunas (omite CPF) e as linhas (apenas ativos).

✅ Validação: Teste de Negação

Na próxima etapa, você testará se o usuário `relator\_visitas` consegue acessar a tabela base `Clientes`. O resultado esperado é **ERRO (Acesso Negado)**.

Etapa 4: Teste de Acesso à View (Hipótese de Sucesso)

 Instrução: Logue como o usuário 'relator_visitas' e execute o comando SELECT na View. O resultado esperado é ****SUCESSO****.

1 Login como 'relator_visitas'

Abra seu cliente SQL (MySQL Workbench, DBeaver, etc.) e faça login com as credenciais:

```
-- Conexão como usuário de baixo privilégio
User: relator_visitas
Password: senha_teste
Host: localhost
```

2 Execute o SELECT na View

Execute o comando abaixo para consultar os dados da View:

```
SELECT * FROM Clientes_Ativos_Simples;
```

 Resultado Esperado: **SUCESSO**

O que você verá: Uma tabela com as colunas `id\_cliente`, `nome` e `status`, mostrando apenas os clientes com `status = 'ativo'`. A coluna sensível `cpf` ****não aparece****, pois a View foi definida sem incluir essa coluna.

Etapa 4: Teste de Acesso à Tabela Base (Hipótese de Negação)

⊘ Instrução: Ainda logado como 'relator_visitas', tente acessar a tabela base e dados sensíveis. O resultado esperado é ****ERRO (Acesso Negado)****.

1 Tentar Acessar a Tabela Base

Tente acessar a tabela base `Clientes` diretamente (sem usar a View). O usuário `relator\_visitas` não tem permissão na tabela base.

```
SELECT * FROM Clientes;
```

⊘ ✖ ERRO - Acesso Negado

2 Tentar Acessar Dados Sensíveis (CPF)

Tente acessar o campo `cpf` da tabela base. Mesmo que o usuário tivesse acesso à tabela, a View não incluiu esse campo.

```
SELECT nome, cpf FROM Clientes;
```

⊘ ✖ ERRO - Acesso Negado

✓ **Validação de Segurança:** Os erros acima são o ****resultado esperado****! Eles provam que a View está funcionando como um filtro de segurança. O usuário `relator\_visitas` não consegue acessar a tabela base nem os dados sensíveis (CPF), mesmo tentando. Isso é o ****Princípio do Mínimo Privilégio (PoLP)**** em ação.

Discussão e Conclusão: Filtro Vertical e Horizontal

🔍 Perguntas-Chave para Discussão

❓ O usuário ``relator_visitas`` conseguiu ver os dados sensíveis (CPF) da tabela base, mesmo que o campo exista na tabela original?

Resposta: Não! Porque a permissão foi concedida **apenas na View**, e a View não incluiu o CPF na sua definição. Este é o **Filtro Vertical** em ação.

❓ O usuário ``relator_visitas`` conseguiu ver os clientes inativos?

Resposta: Não! Porque a View filtrou os dados com ``WHERE status = 'ativo`` antes da concessão de acesso. Este é o **Filtro Horizontal** em ação.

💡 Conclusão: O Poder das Views

📄 Filtro Vertical (Colunas)

A View **omitiu o CPF** (coluna sensível). O usuário só vê as colunas que a View expõe: ``id_cliente`, `nome`, `status``. Restrição de **colunas**.

⚡ Filtro Horizontal (Linhas)

A View **filtrou apenas clientes ativos** (linhas com ``status = 'ativo``). O usuário não vê clientes inativos ou pendentes. Restrição de **linhas**.

✅ PoLP Reforçado com Sucesso

As Views não apenas **simplificam** consultas complexas. Elas atuam como um **filtro de segurança** que reforça o **Princípio do Mínimo Privilégio (PoLP)** de forma eficiente e elegante.

👥 Views: Ponto de Acesso Principal

Em ambientes profissionais, analistas de dados e aplicações **raramente acessam as tabelas base diretamente**. Eles acessam Views ou Data Marts criados especificamente para seu propósito, garantindo **performance** e **segurança**.

Encerramento: Registro de Resultados e Próximos Passos

📝 Registro de Resultados (Bloco 4)

</> 1. Comando CREATE VIEW

Registre o comando ``CREATE VIEW Clientes_Ativos_Simples AS...`` que você executou. Este é o primeiro passo da política de segurança.

🔑 2. Comando GRANT

Registre o comando ``GRANT SELECT ON Clientes_Ativos_Simples TO 'relator_visitas'@'localhost'``. Prova de que a permissão foi concedida apenas na View.

⊗ 3. Mensagem de ERRO

Registre a mensagem de erro obtida ao tentar acessar a tabela base ``Clientes`` como ``relator_visitas``. Exemplo: ``Access denied for user 'relator_visitas'@'localhost'``.

✔ Prova de Segurança

O registro desses três elementos (CREATE VIEW, GRANT, ERRO) serve como ****prova de que a política de segurança foi validada com sucesso**** e é essencial para auditorias internas e externas.

➡ Síntese e Próximos Passos

💡 Síntese: Views em Ambientes Profissionais

Em ambientes profissionais, ****analistas de dados e aplicações raramente acessam as tabelas base diretamente****. Eles acessam Views ou Data Marts criados especificamente para seu propósito, garantindo a ****performance e a segurança de dados sensíveis****.

📖 Próximo Passo: Mapeamento do Fluxo de Dados (Data Pipeline)

Na próxima aula (Aula 08), vamos analisar como os dados fluem através do sistema, desde a origem até o consumidor final. Entenderemos o conceito de ****Data Pipeline**** e como as Views se encaixam nesse fluxo.

📅 Preparação para a Próxima Aula:

Revisar os conceitos de ETL/ELT, as camadas Bronze, Prata e Ouro do Data Lake, e como as Views são utilizadas em cada camada para garantir segurança e performance.